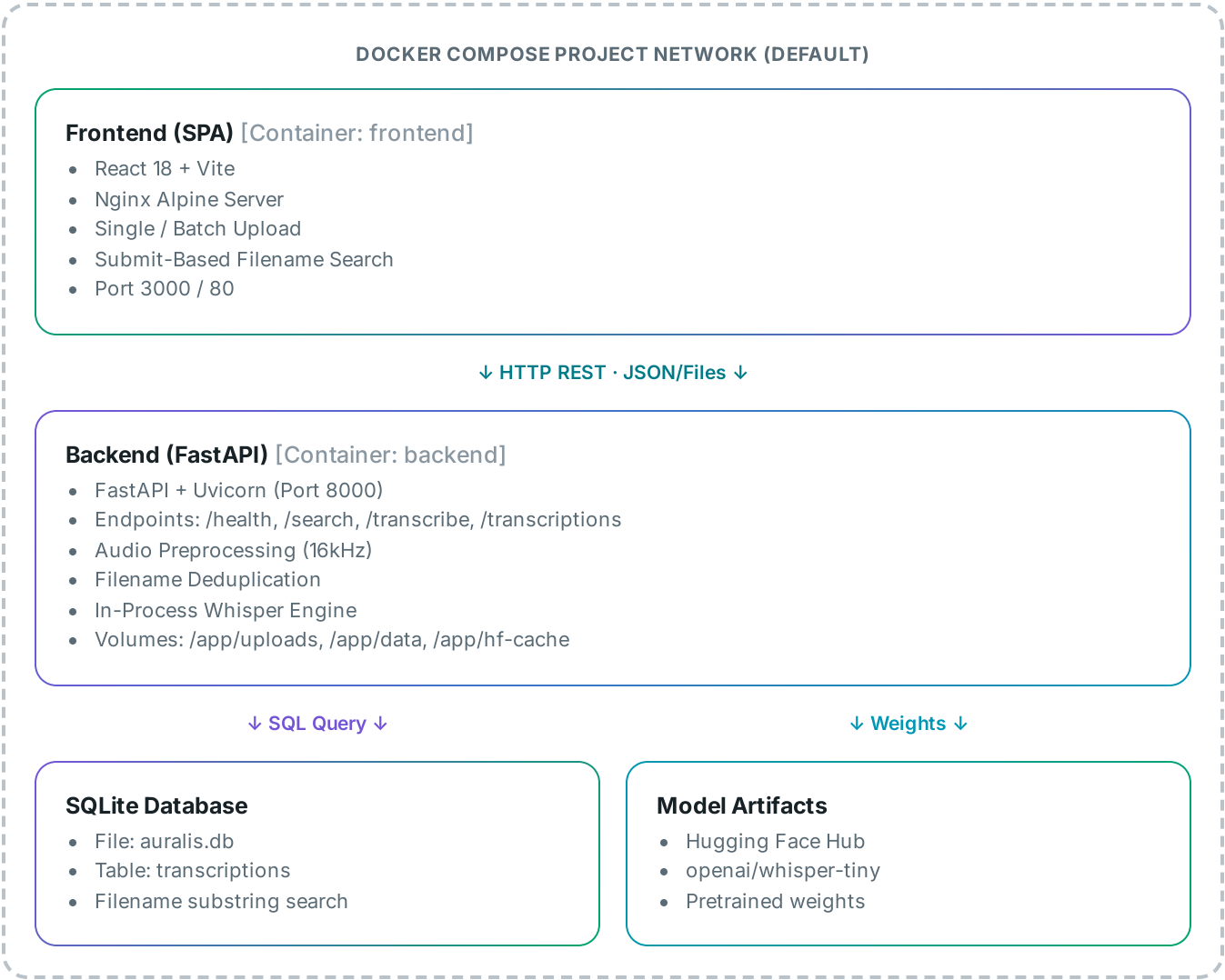


# Auralis — System Architecture & Technical Specifications

Project: Auralis Speech-to-Text Platform | Model: openai/whisper-tiny | Repository: [github.com/yingmusee/auralis](https://github.com/yingmusee/auralis)

"Auralis" is an assumed project name adopted for this build — the original brief did not specify one.

## 1. System Architecture Diagram



## 2. Architectural Components Breakdown

### a) Frontend (Single-Page Application)

- **Stack & Build:** React 18, Vite development server and production build system, HTML5 Drag & Drop File API, CSS responsive styling.

- **Capabilities:** Single & batch audio upload via drag-and-drop or file picker with a real-time upload progress bar, persistent transcription records table, and client-to-server filename search.
- **Containerization:** Vite compiles the React app during the multi-stage image build; the final Alpine Nginx container serves static assets on port 80 (mapped to host 3000). Local development instead runs the Vite server with hot reload on port 5173.
- **Assumptions & Considerations:** Search is deliberately submit-based (a button or Enter key) rather than firing a request on every keystroke, since the backend has no debounce or query-cancellation support and is otherwise rate-limited — a live-search UI would need both first. The backend URL the compiled frontend calls (`VITE_API_BASE_URL`) is a build-time argument, not a runtime setting, so deploying the frontend and backend to different hosts means rebuilding the frontend image with the right value. The upload panel accepts a broad set of audio containers by file extension as well as MIME type, since browsers report inconsistent or generic MIME types (e.g. `application/octet-stream`) for less common formats like AIFF, WMA, and MKA.

## b) Backend (RESTful Web Service)

- **Engine & Framework:** FastAPI (Python 3.11) with Uvicorn ASGI, asynchronous request handling, and CPU-bound Whisper inference offloaded to a worker thread (`asyncio.to_thread`) so it doesn't block the event loop.
- **Endpoints:** `GET /health` (liveness probe), `POST /transcribe` (multipart single/batch ingestion with deduplication), `GET /transcriptions` (chronological record retrieval), `GET /search?filename=...` (filename substring query).
- **Audio Preprocessing Pipeline:** Decodes uploaded formats (MP3/WAV/M4A/AAC/FLAC/OGG/Opus/WebM/MKA/AIFF/AIF/WMA), using `ffmpeg` for containers that `soundfile` cannot read, resamples audio to 16,000 Hz, downmixes stereo to mono, and normalizes amplitudes to float32 tensors per Whisper specification.
- **Filename Deduplication:** Atomically reserves upload paths and checks SQLite for filename collisions; appends a unique random UUID suffix (`{stem}_{uuid[:8]}{ext}`) while preserving the original user-facing filename.
- **Assumptions & Considerations:** The API surface is deliberately limited to the four endpoints above; FastAPI's automatic interactive docs (`/docs`, `/redoc`, `/openapi.json`) are disabled rather than left enabled as an unspecified extra. With no authentication in front of `/transcribe`, the endpoint is hardened against trivial resource exhaustion instead: a 25MB per-file limit, a 10-files-per-request cap, a 250MB total-request-body ceiling enforced at the ASGI layer (ahead of FastAPI's own parsing, which does not itself cap individual file part size), and a 10-requests/minute per-client rate limit. A `MOCK_WHISPER` environment variable substitutes a placeholder transcript for real model inference; the automated backend test suite runs with this set so tests are fast and deterministic. Whisper inference is offloaded to a worker thread per file so the event loop stays free for unrelated concurrent requests; files within one batch are still transcribed one at a time against the shared in-memory model, since real parallel speedup would need a bounded worker pool and is capped by available CPU cores. `POST /transcribe` standardizes on returning an array of per-file JSON objects (`id`, `filename`, `original_filename`, `transcript`, `created_at`) for both single and batch uploads, so callers don't need two response shapes.

## c) Database (SQLite)

- **Storage:** SQLite 3 relational database (auralis.db) mounted on persistent Docker volume (backend-data).
- **Schema & Indexing:** Table transcriptions stores id (PK), filename (UNIQUE), original\_filename, transcript, and created\_at (ISO 8601 UTC). filename's UNIQUE constraint gives it SQLite's automatic B-Tree index, which is what the exact-match filename-collision check actually uses. No other index is defined: /search's own query is a full table scan regardless, since SQLite can't use a B-Tree index for a LIKE pattern with a leading wildcard (%query%) — an explicit index on original\_filename would add write overhead for zero read benefit.
- **Assumptions & Considerations:** SQLite was chosen because the spec asks for it directly, and its embedded, single-file nature matches this project's single-instance scope (see Section 3). Audio files are saved to uploads/ while metadata and transcriptions reside in SQLite; both are mounted via Docker named volumes to survive restarts and rebuilds. The filename UNIQUE constraint is the actual safety net against duplicate names, not just the application-level check — two concurrent uploads can still race to reserve the same candidate name, and the constraint (with an app-level retry-under-new-name) is what guarantees no two records ever end up sharing one.

#### d) Model Artifacts & Integration

- **Model Hub:** Hugging Face Hub (openai/whisper-tiny, ~151 MB weights). The model is pre-warmed during FastAPI's startup lifespan, before the container reports healthy, so the first real transcription request never pays the load cost.
- **Runtime Dependency:** The Hugging Face cache directory is mounted on a persistent Docker volume (backend-hf-cache), so weights are downloaded once and reused across restarts; inference and audio processing run locally inside the backend container.
- **Assumptions & Considerations:** Whisper-tiny runs directly within the FastAPI backend worker process rather than behind a separate inference microservice. Given the compact model footprint (39M parameters, ~151MB), in-process inference eliminates network overhead and simplifies deployment into a single backend container; a larger Whisper variant would likely need to move inference out of the request-handling process instead. Audio formats libsndfile can't decode (e.g. M4A/AAC) are routed through an ffmpeg subprocess fallback rather than rejected outright — this assumes support for a wider set of common containers beyond a single format, since the brief didn't specify which ones to handle.

### 3. Production Scale Considerations

The assumptions behind each component's current implementation are covered alongside that component in Section 2. This section looks forward: how the current implementation's main limits would be addressed moving from a single-instance exercise toward production load.

| Domain                           | Current Implementation                                                                                                                                                    | Production Scale Recommendation                                                                                                                        |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Inference Concurrency</b>     | CPU inference per file, offloaded to a worker thread so it doesn't block the event loop; files within one batch upload still transcribe one at a time.                    | Offload transcription to an asynchronous Celery / Redis task queue with WebSocket status streaming to prevent HTTP timeouts on long audio files.       |
| <b>Storage Architecture</b>      | Local container filesystem mounted to a Docker named volume.                                                                                                              | Migrate audio storage to cloud object storage (AWS S3, GCP Cloud Storage) with presigned URLs for direct client uploads.                               |
| <b>Database Scaling</b>          | Embedded SQLite; exact-match filename lookups use an index, but /search's substring query is a full table scan (a B-Tree index can't accelerate a leading-wildcard LIKE). | Migrate to PostgreSQL or a distributed relational database with trigram/full-text search (pg_trgm, Elasticsearch) for filename and transcript content. |
| <b>Security &amp; Compliance</b> | 25 MB per-file upload limit, in-memory per-client rate limiting, path-safe filename handling, UUID collision isolation, and configurable CORS origins.                    | Add durable distributed rate limiting, JWT bearer authentication, audio malware scanning (ClamAV), and HTTPS TLS termination at reverse proxy.         |

### 4. Verification & Testing Matrix

- **Backend Unit Tests:** Three pytest tests in backend/tests/test\_api.py covering the four required endpoints: a health check, an upload that returns a transcription and deduplicates on filename collision and appears in the transcriptions list, and a filename search.
- **Frontend Unit Tests:** Three Vitest/RTL tests in frontend/src/App.test.jsx covering the three required UI features: transcription table rendering, upload with progress, and filename search.
- **Local Setup:** Run setup-local.sh from the repository root to install backend and frontend dependencies, or run backend/setup-backend.sh to prepare only the Python backend environment.